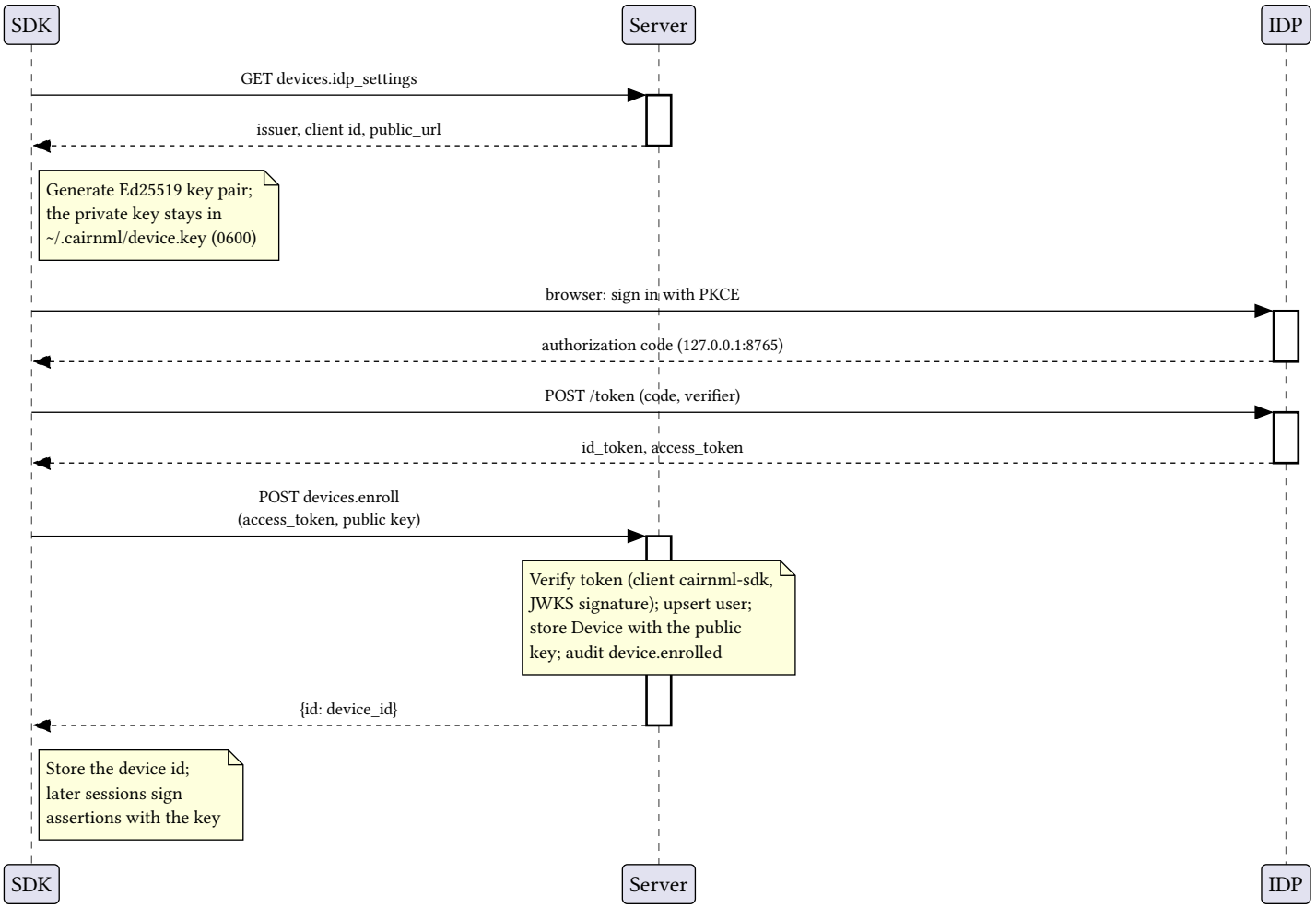
CairnML: sequence diagrams

Common operations between the SDK, the server, the identity provider, the agent, the web app, object storage and Git.

Participants: **SDK** (Python client), **Server** (CairnML API), **IDP** (Keycloak), **Agent** (job runner), **WebUI** (browser app), **RustFS** (object storage), **Git** (source repositories). Solid arrows are requests and dashed arrows are replies. Grey notes are processing inside a party. Endpoint names are those in the code.

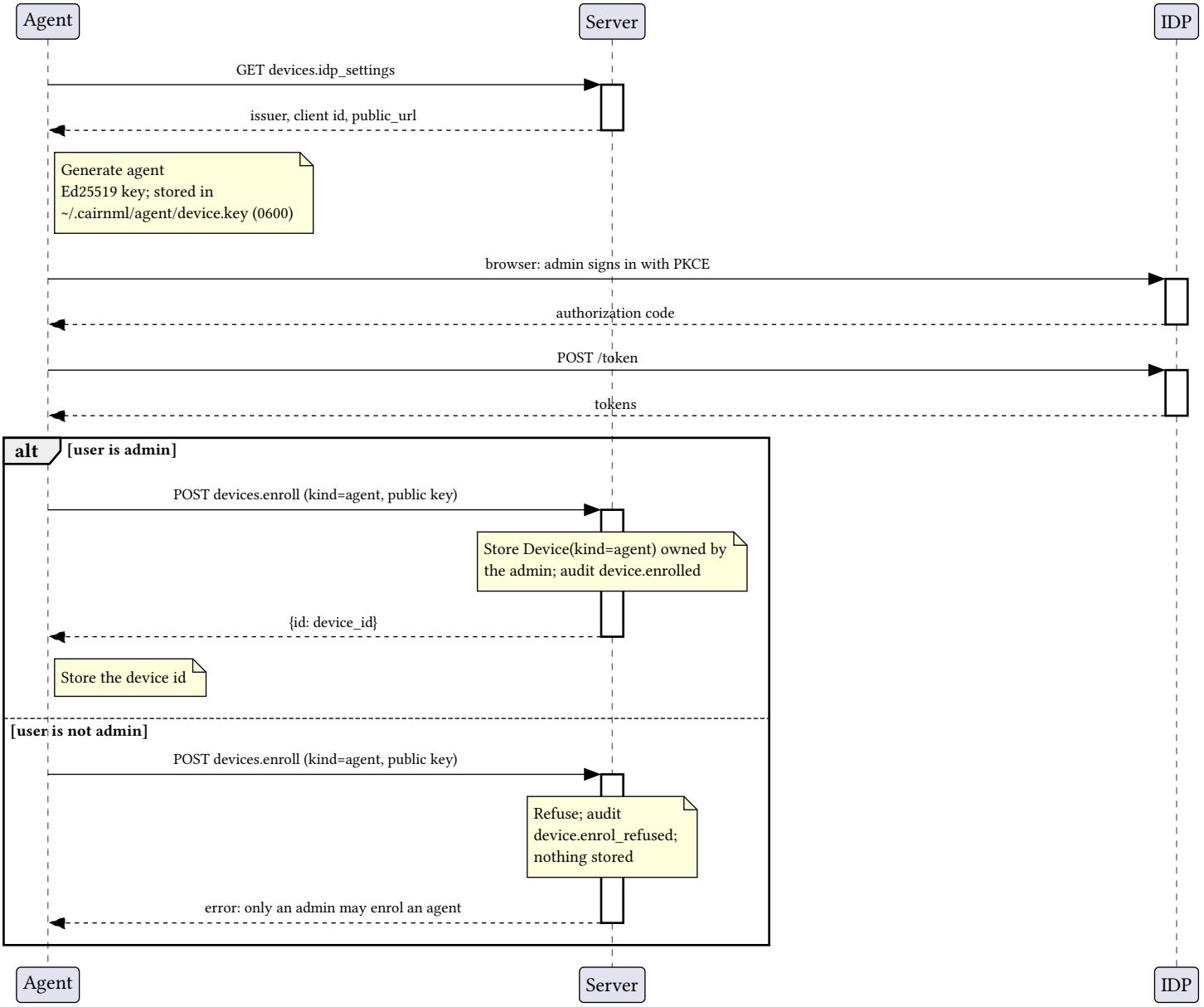
1. Enrollment

1.1. SDK enrollment (`cairnml init <server>`)



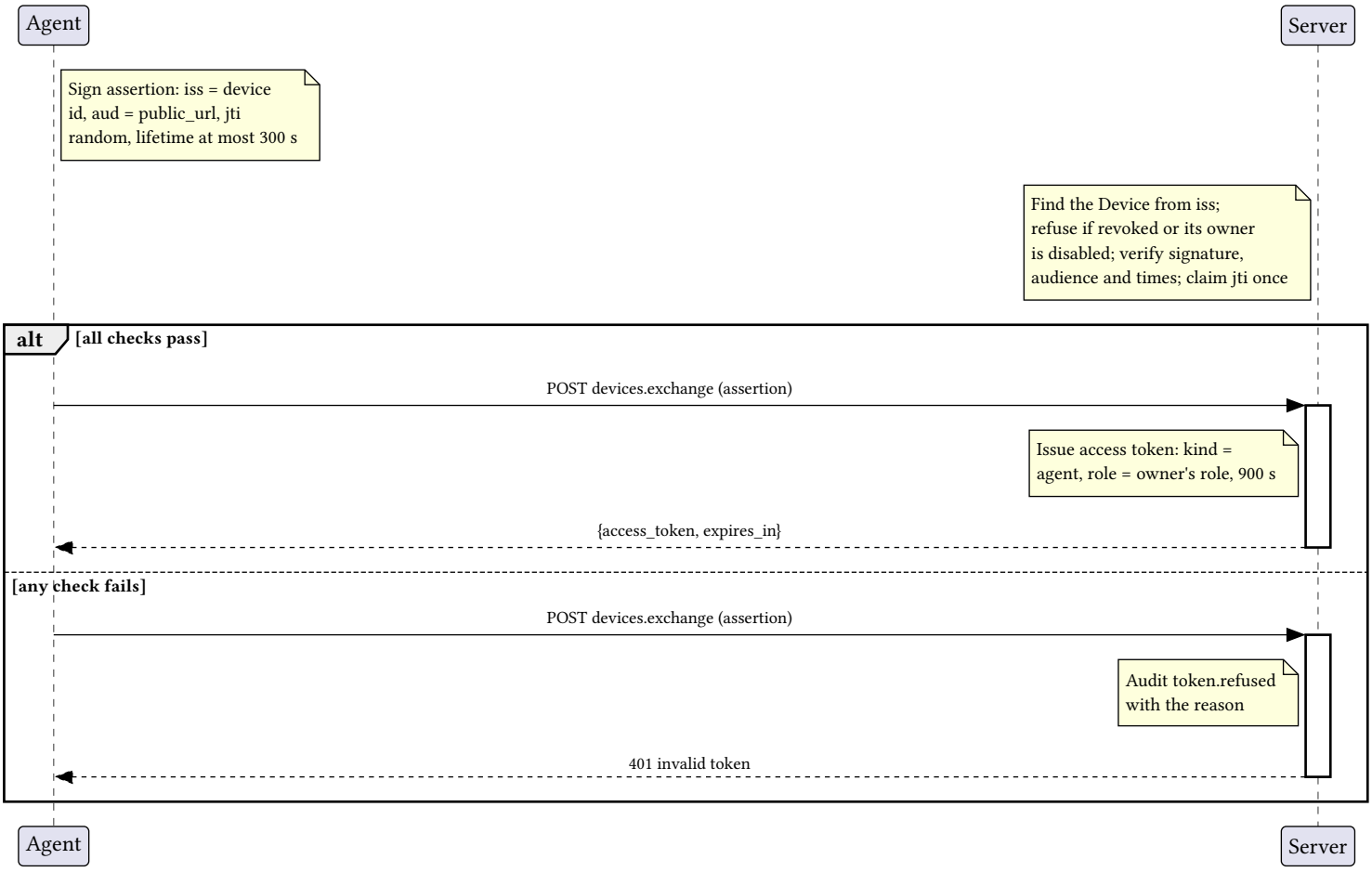
1.2. Agent enrollment (`cairnml-agent init <server>`)

The same handshake with the agent's own key and kind `agent`. The server refuses unless the signed-in user is an admin.



1.3. Device token exchange (every SDK or agent session)

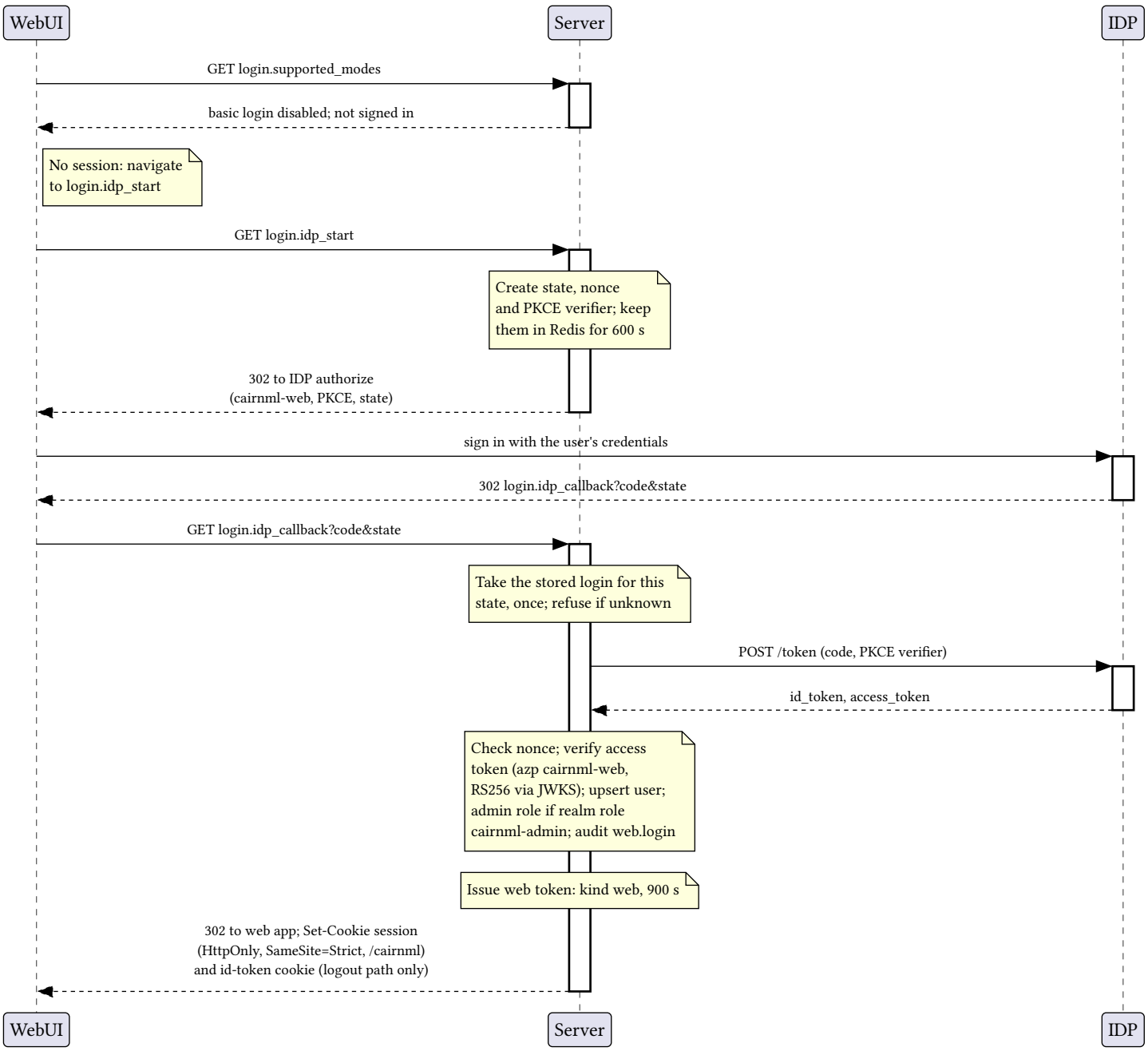
The device proves possession of its key with a short signed assertion, which can be used once. The server returns a 15-minute access token that carries the owner's role.



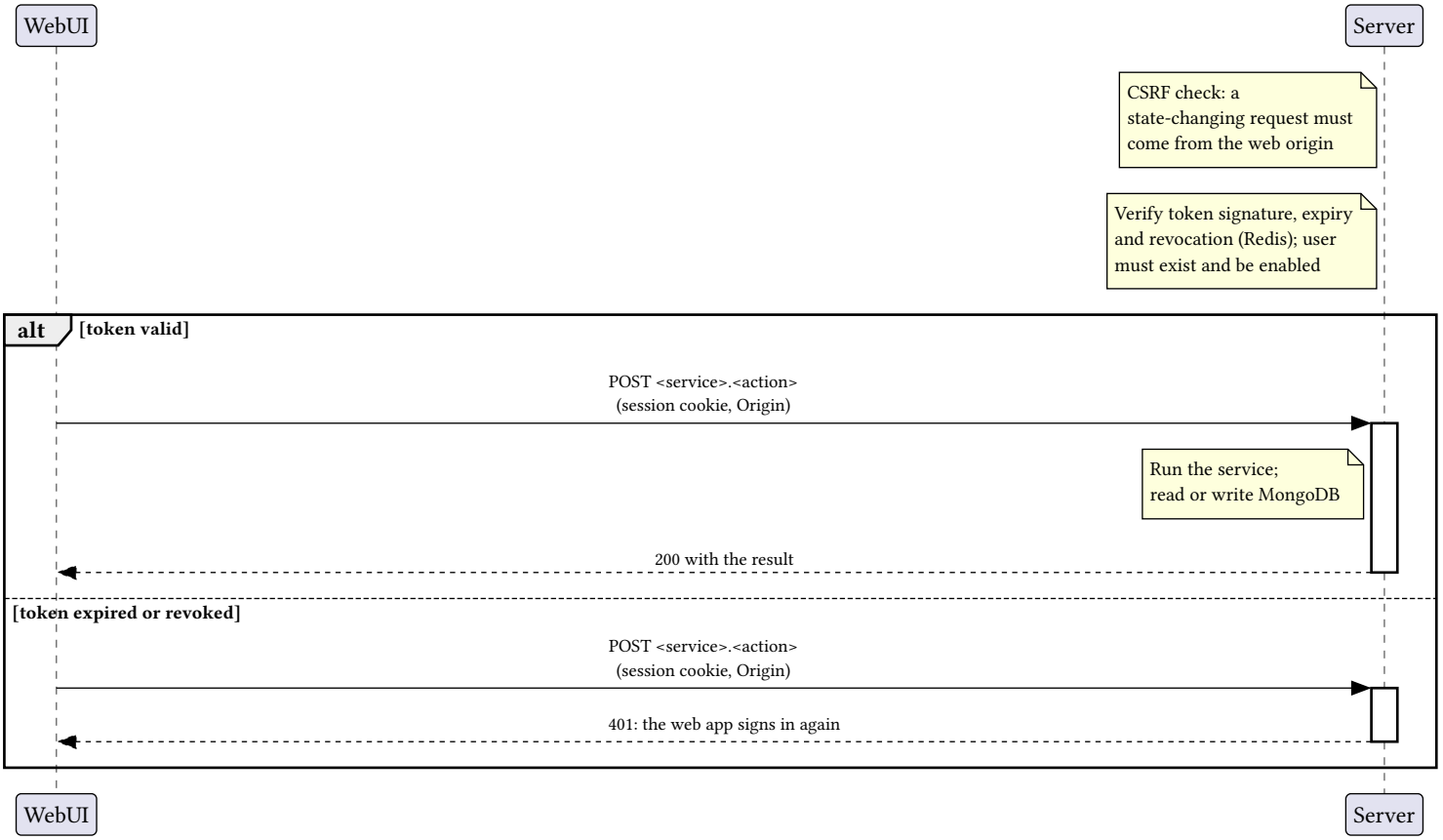
The web app holds no credential of its own. The browser goes to the identity provider; the server then sets an HttpOnly session cookie and a second cookie with the ID token, which is sent only to the logout endpoint.

2. WebUI login and session

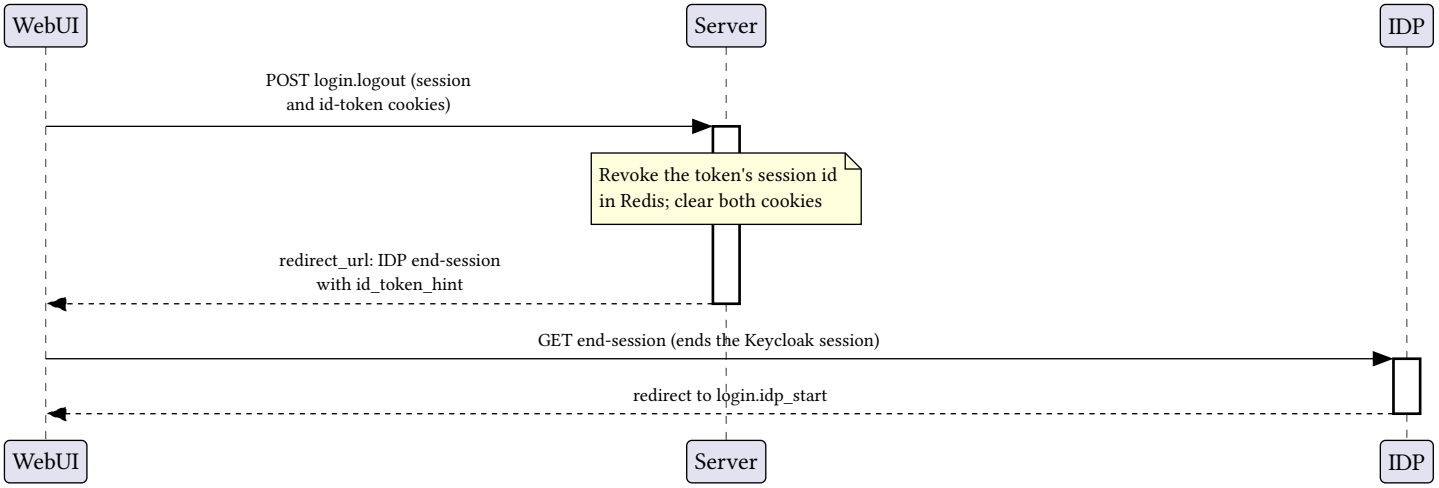
2.1. Sign-in (authorization code with PKCE)



2.2. Authenticated request



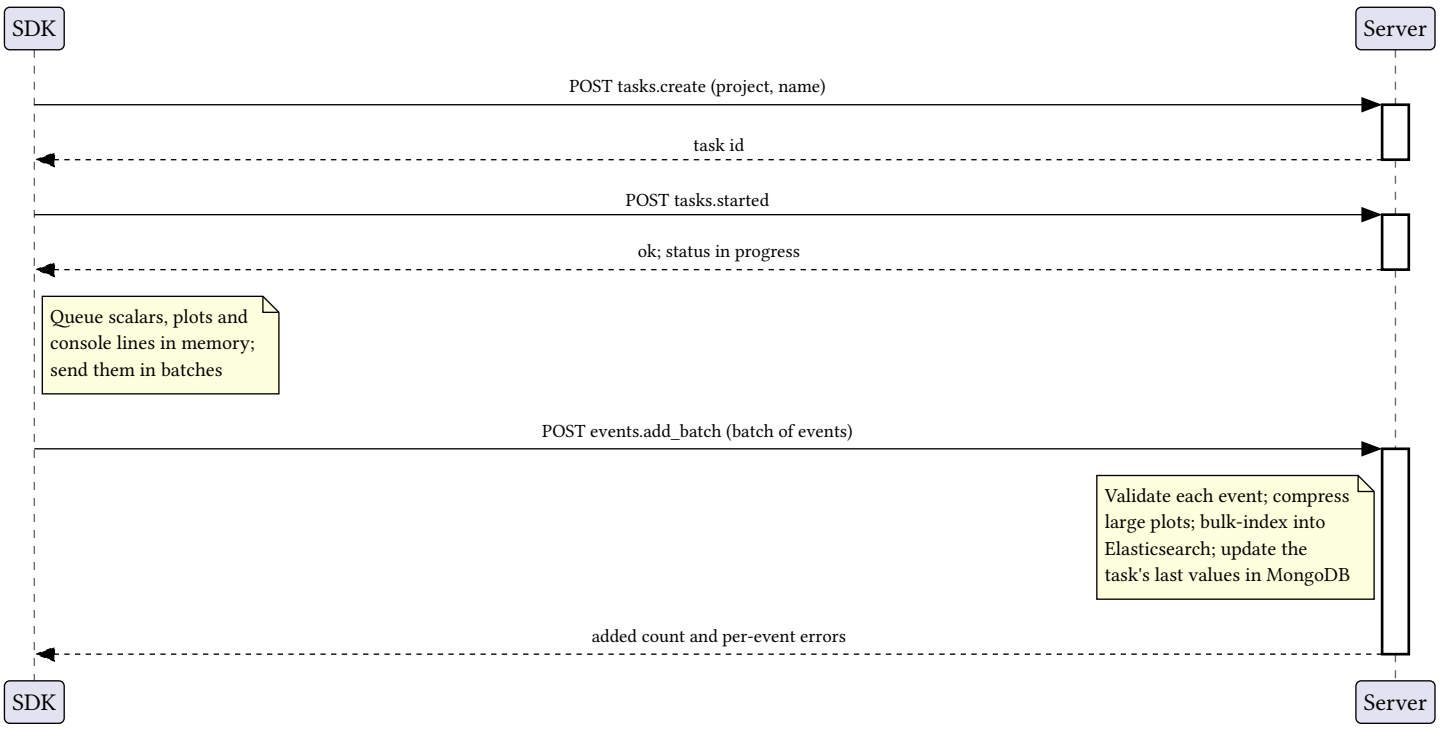
2.3. Logout



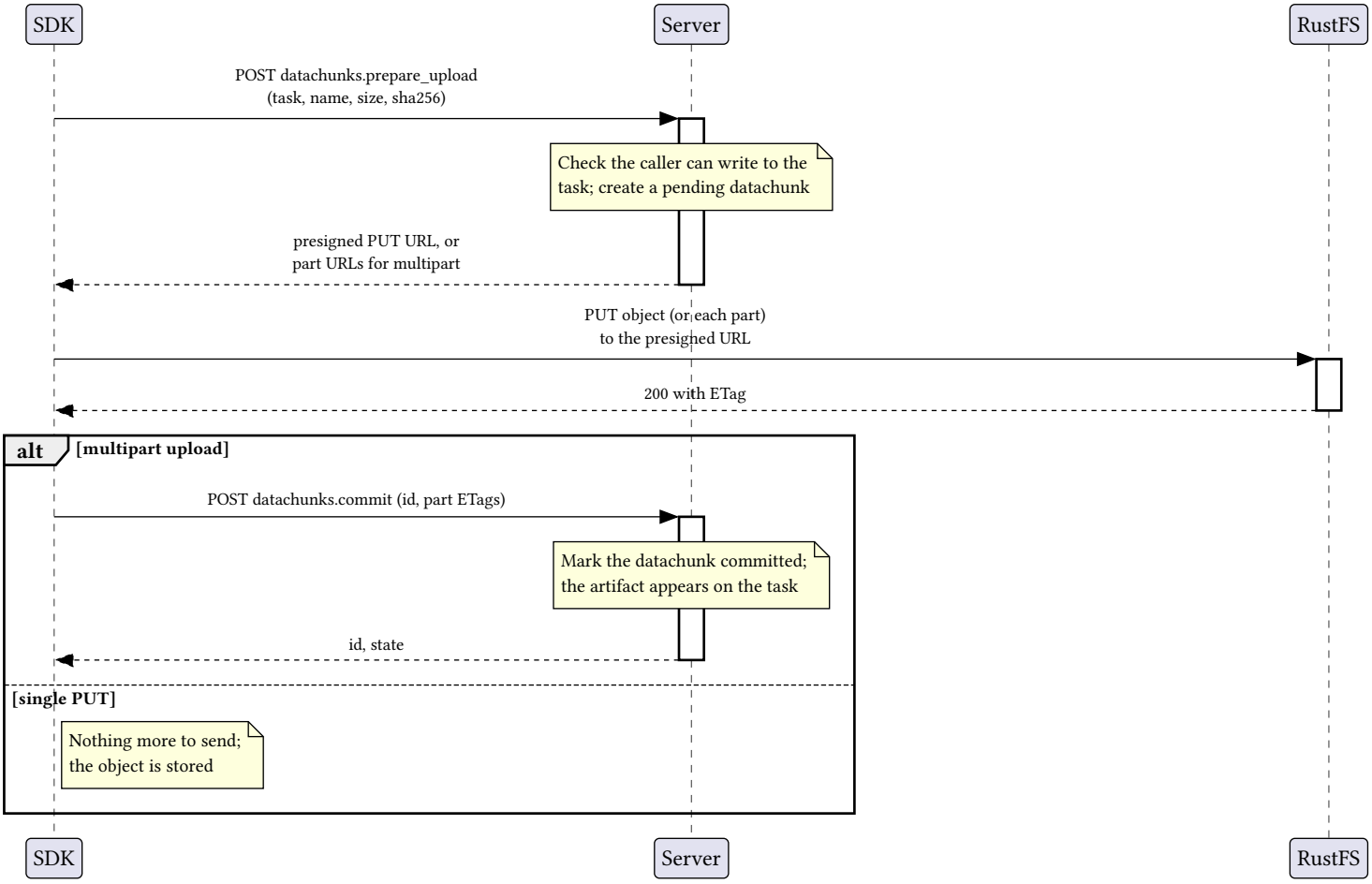
Events are batched and sent to the server. Large files do not pass through the API: the server returns a presigned object-storage URL and the SDK uploads straight to RustFS.

3. Task logging

3.1. Creating the run and sending events



3.2. Artifact upload (presigned, direct to object storage)

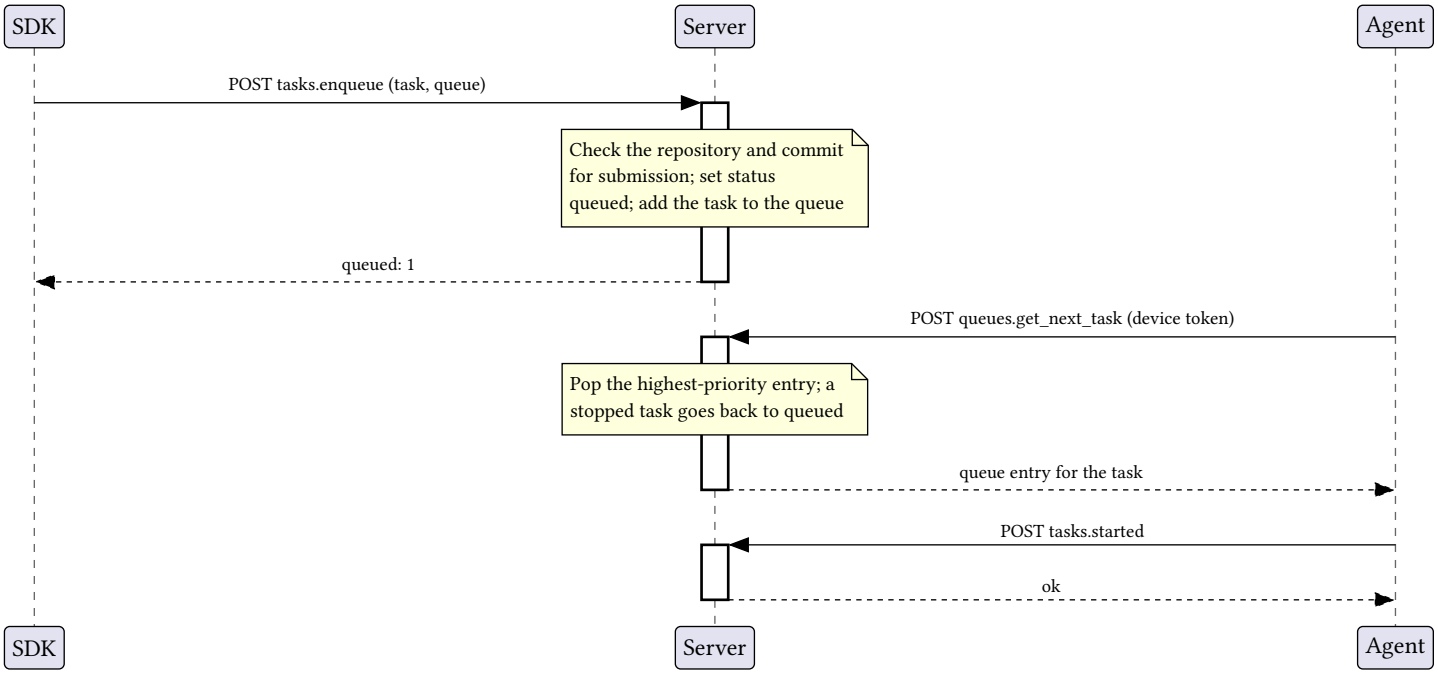


Console lines are sent in the same `events.add_batch` call as scalars, as the `log` event type, and appear on the task's Console tab.

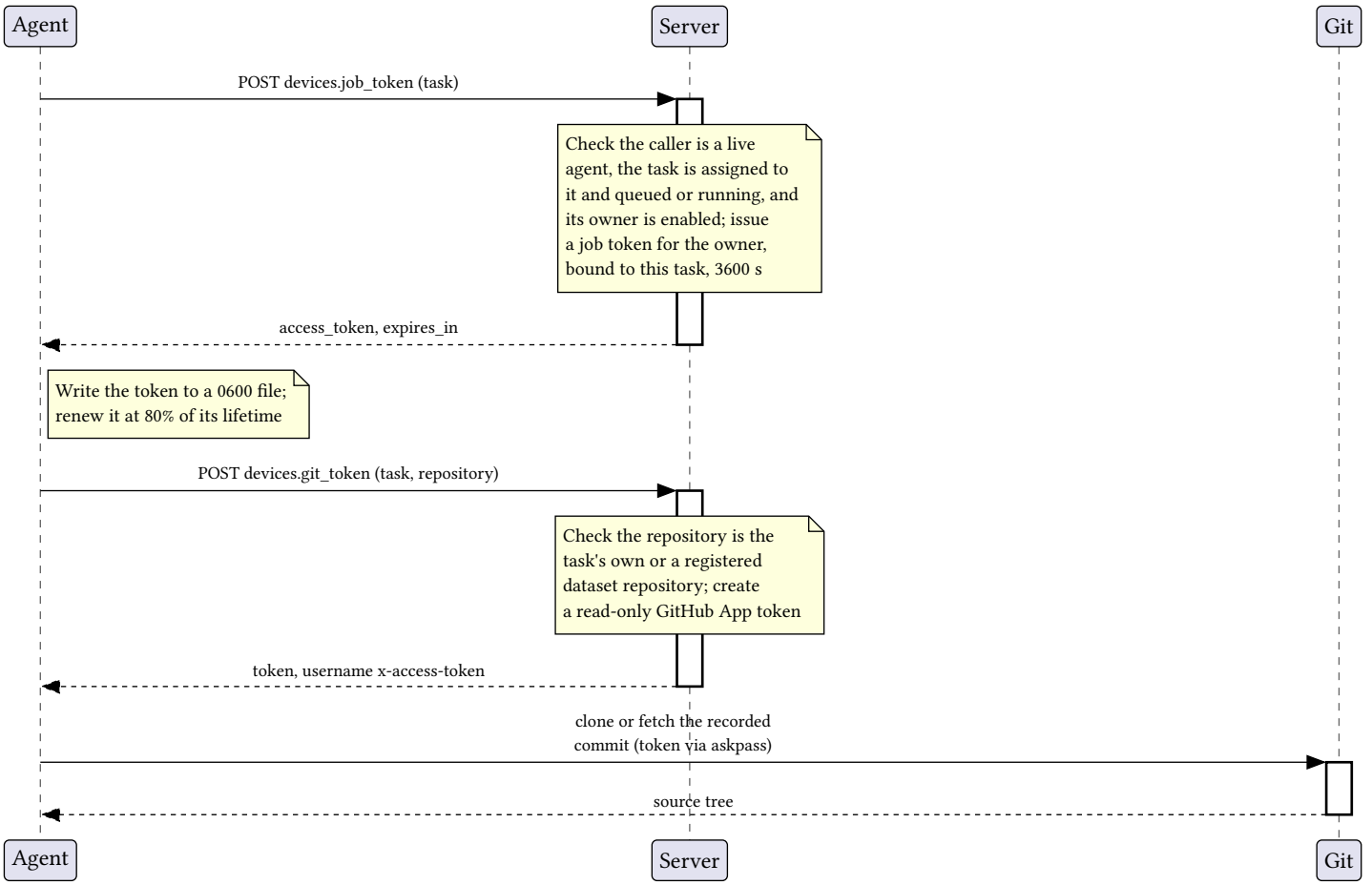
The SDK or the web app puts a task in a queue. An agent pulls it, obtains a token for this job alone, fetches the source with a server-issued Git credential, runs it and reports the result. The job never uses the agent's own token.

4. Job execution on an agent

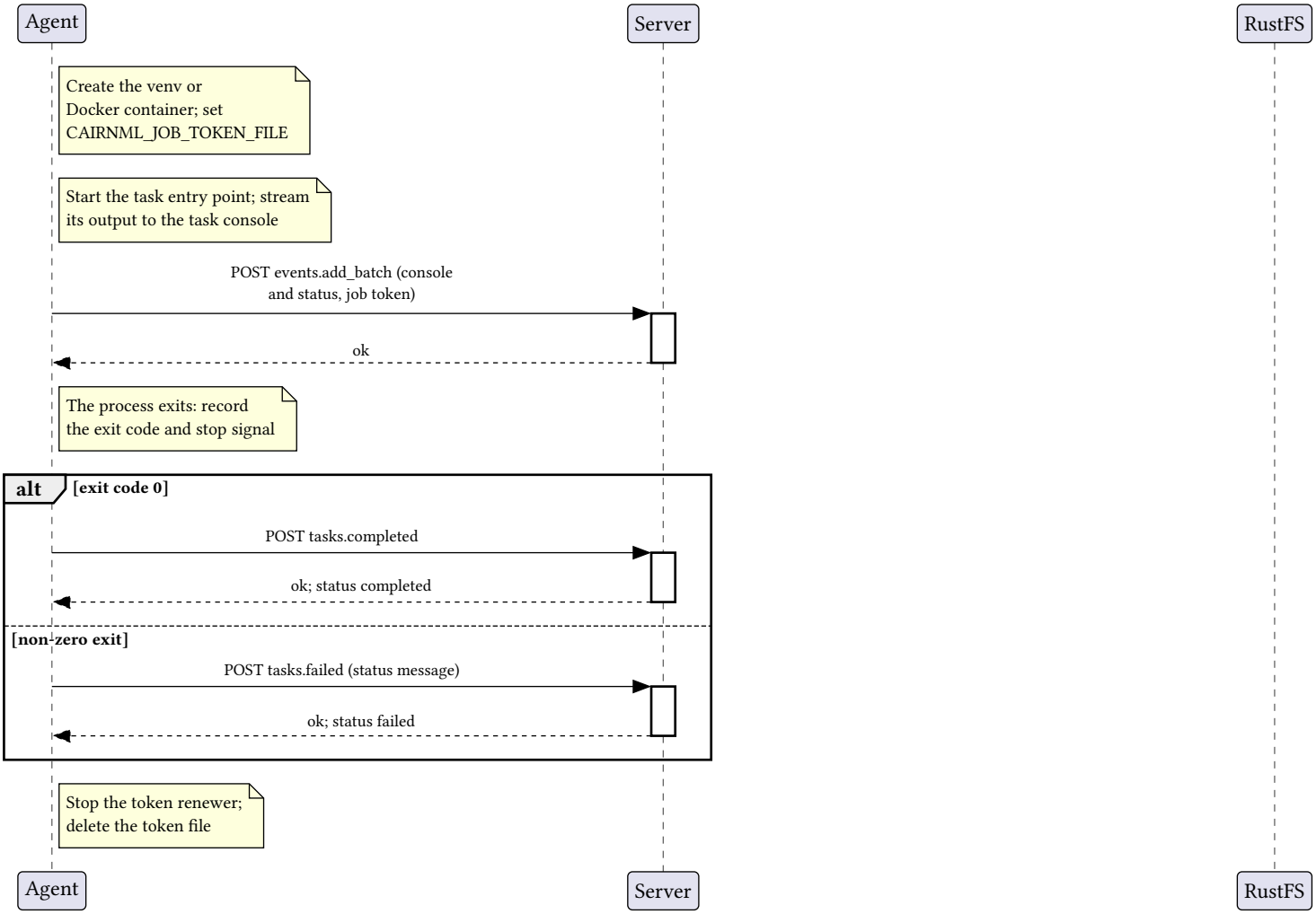
4.1. Enqueue and pull



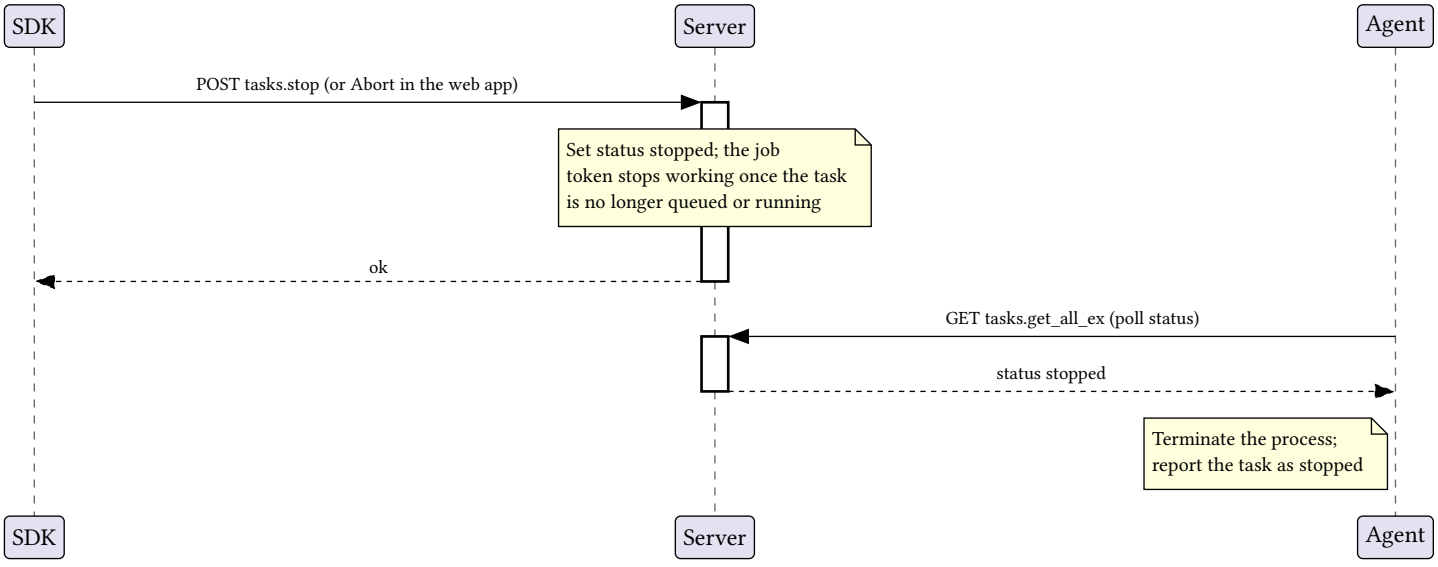
4.2. Job token and source



4.3. Running and reporting



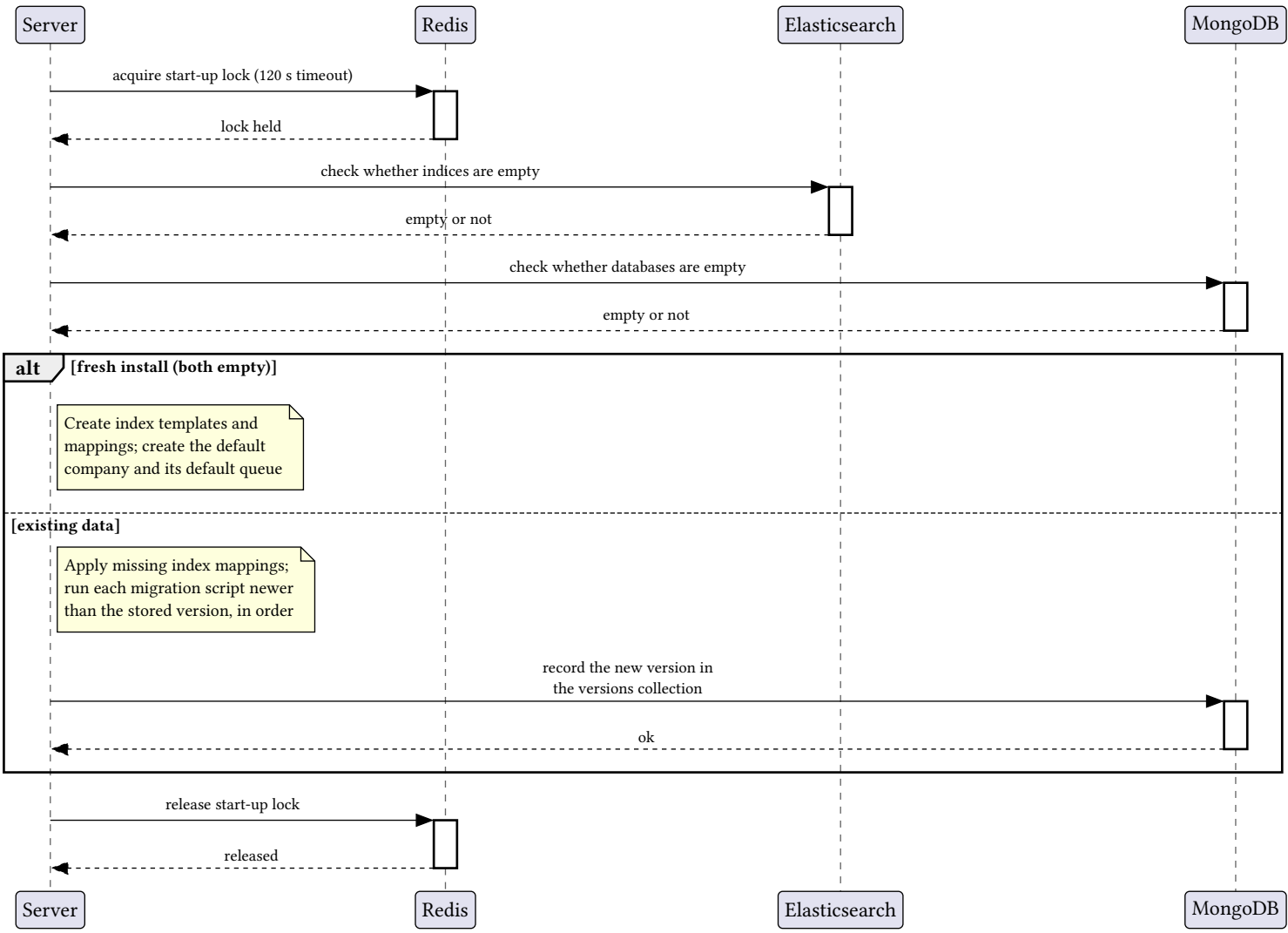
4.4. Abort



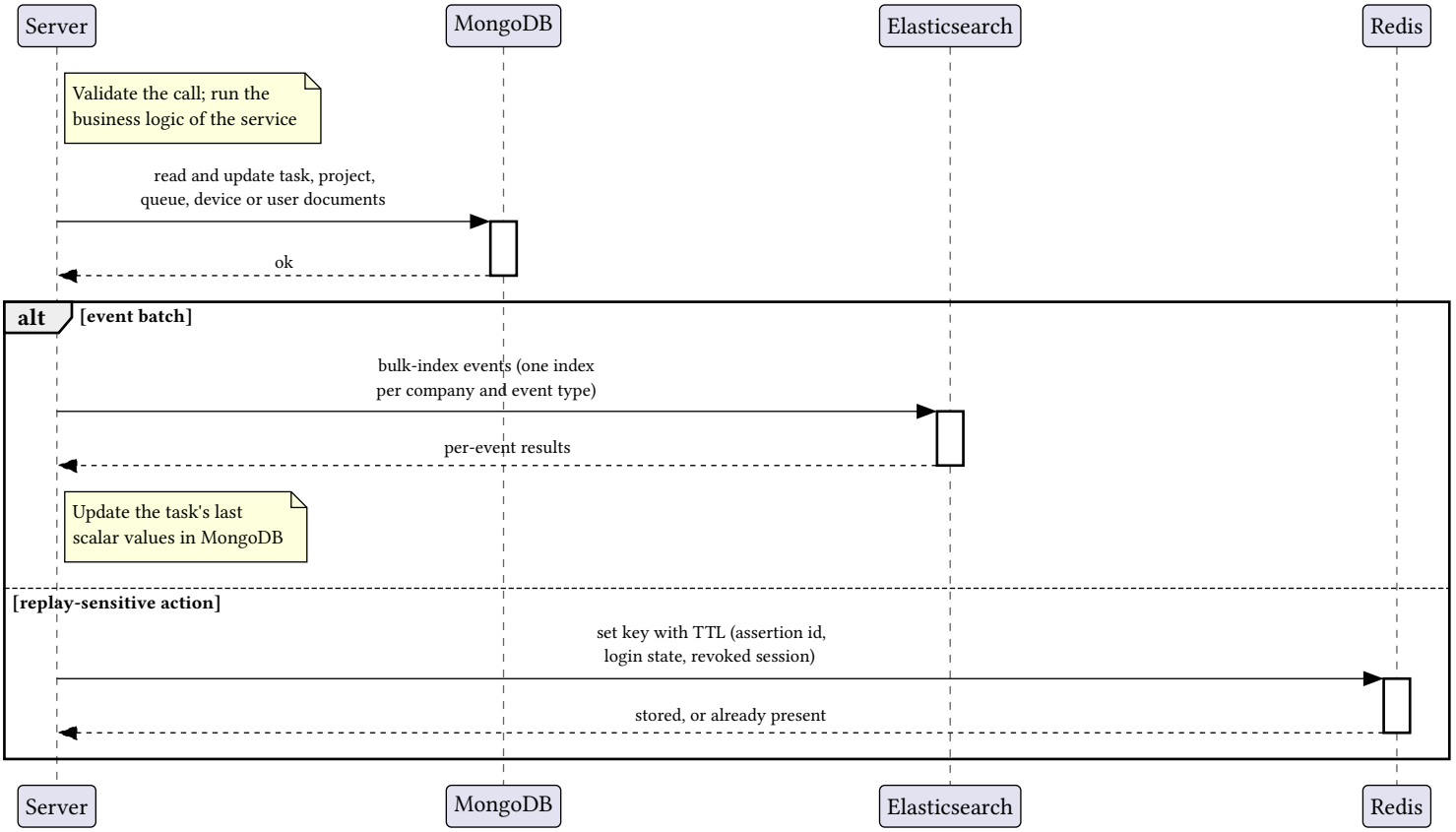
The server keeps MongoDB (two databases: `backend-db` for tasks, models and projects, and `auth-db` for access records), Elasticsearch (event and metric indices) and Redis (short-lived state and a start-up lock). Object bytes live in RustFS, never in a database.

5. Database management

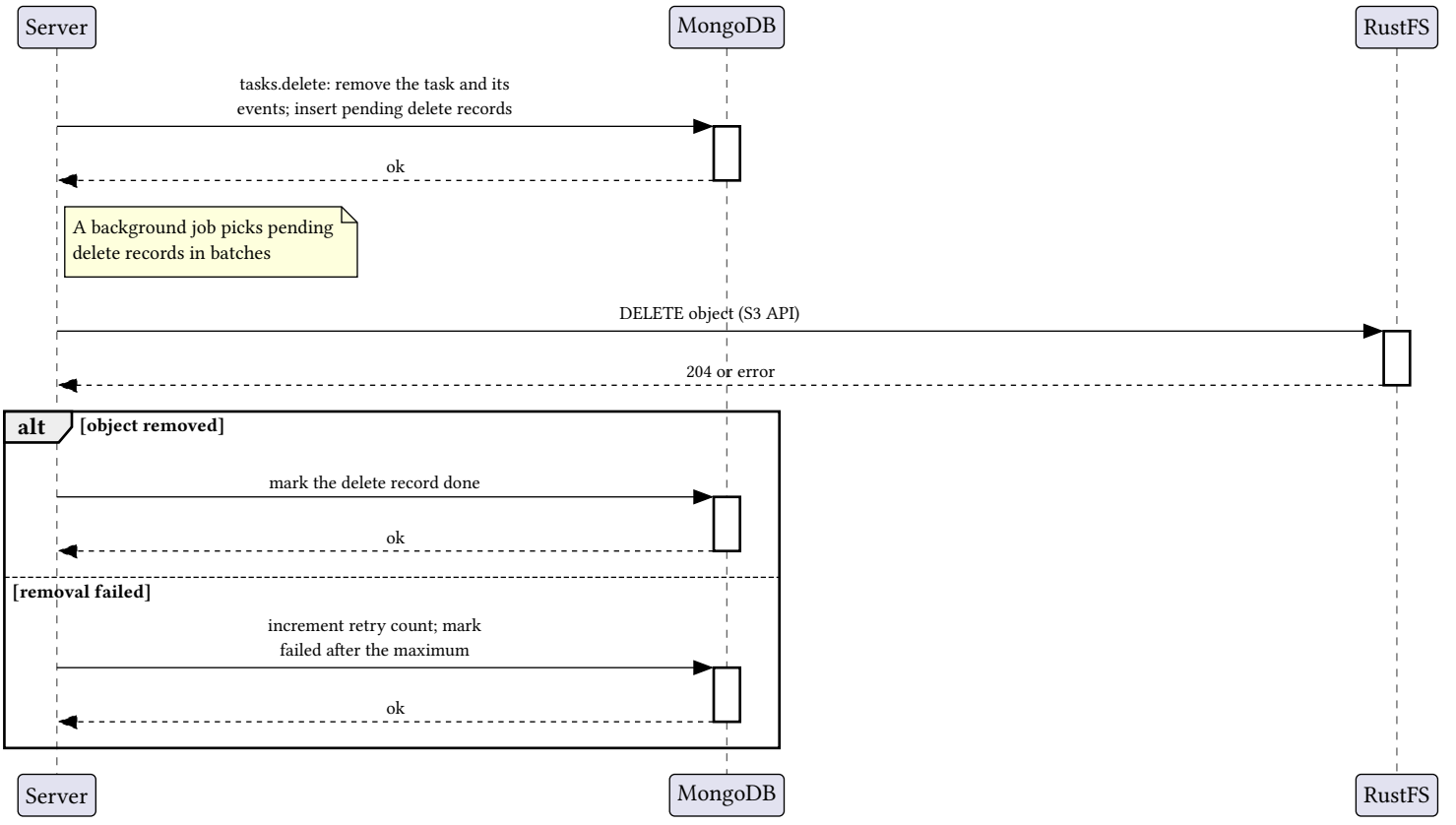
5.1. Start-up: initialise or migrate



5.2. Record writes



5.3. Removing objects (asynchronous delete)



5.4. Schema migrations

Each release that changes stored data ships a migration script in `apiserver/mongo/migrations/`. Scripts run at start-up, only when the databases already contain data.

